

Unity를 이용한 2D 게임프로그래밍

동명대학교 게임학부

Lecture 9

Unity 스크립트로서의 C#

일반 C# 프로그램과 Unity 스크립트의 차이

강영민

동명대학교 게임학부

대상 독자

이미 일반 C# 프로그래밍의 기초(클래스, 상속, 메서드, 제어문, 컬렉션 등)는 알고 있는 학생. Unity는 처음 접하더라도, “같은 C# 언어인데 왜 작성 방식이 이렇게 달라 보이는가”에 답을 얻는 것이 목표이다.

실습 환경: Unity Editor + Visual Studio Code (또는 Visual Studio)

Contents

1 강의 개요	3
1.1 학습 목표	3
1.2 출발점: 일반 C#으로 게임을 만들 때 우리가 직접 한 일들	3
1.3 프레임워크와 라이브러리의 차이	4
2 Unity 스크립트의 기본 구조	4
2.1 가장 기본적인 Unity 스크립트	4
2.2 MonoBehaviour란 정확히 무엇인가?	5
2.3 일반 C# 프로그램과의 차이점	5
2.4 Scene(씬)이란?	6
3 MonoBehaviour의 생명 주기	6
3.1 핵심 생명 주기 메서드	6
3.2 Awake와 Start의 차이	6
3.3 Update와 FixedUpdate의 차이	7
3.4 Rigidbody2D와 물리 엔진	7
3.5 Time.deltaTime의 의미	8
4 컴포넌트 기반 설계	8
4.1 GameObject와 Component	8
4.2 상속(Inheritance) vs 조합(Composition)	9
4.3 GetComponent로 컴포넌트 접근	9
4.4 Transform: 모든 GameObject의 필수 컴포넌트	10
5 Inspector와 직렬화	11
5.1 public 필드는 Inspector에 노출된다	11
5.2 권장 패턴: [SerializeField] private	12
5.3 Inspector가 있다는 것의 의미	12
6 Unity 스크립트의 주요 고려 사항	13
6.1 (1) MonoBehaviour는 new로 만들지 마라	13
6.2 (2) Update에서 무거운 작업을 하지 마라	14
6.3 (3) Unity의 null은 일반 C#의 null과 다르다	15
6.4 (4) Coroutine: 시간에 따라 펼쳐지는 작업	15
6.5 (5) 직접 그리지 않는다	16
7 입력과 충돌 처리	17
7.1 한눈에 보는 비교: 일반 C#/GUI vs Unity	17
7.2 Input.GetKey 세 가지 변형	17
7.3 Collider와 충돌 콜백	18
7.4 Unity가 대신 해 주는 것들	19
8 실전 예제: 충돌하면 합쳐지는 공들	19
8.1 게임 컨셉과 물리 규칙	19
8.2 Hierarchy 설계	20
8.3 Prefab 설계: BallPrefab	20
8.4 스크립트 설계: 역할 분리	21
8.5 Ball.cs 구현	21
8.6 BallSpawner.cs 구현	23
8.7 Unity Editor에서 적용하기 (단계별)	24

8.8	동작 분석과 확장	25
9	단계별 도전 과제	25
9.1	Phase 1 — 예제 변형 (기초)	25
9.2	Phase 2 — 새 메커니즘 추가 (응용)	27
9.3	Phase 3 — 새 게임 만들기 (종합 응용)	29
9.4	Phase 4 — 자유 창작	30
10	이번 강의에서 배운 핵심 개념	31

1. 강의 개요

1.1. 학습 목표

같은 C# 언어를 사용하지만, Unity 안에서 동작하는 스크립트는 일반 C# 프로그램과 작성 방식이 매우 다르다. 이번 강의에서는 그 차이점과 주의해야 할 점을 정리한다.

- 일반 C# 프로그램과 Unity 스크립트의 진입 방식 차이를 이해한다.
- MonoBehaviour와 그 생명 주기 메서드(Awake, Start, Update 등)의 호출 시점을 파악한다.
- Unity의 컴포넌트 기반 설계와 GameObject/Component 모델을 이해한다.
- Unity의 직렬화(Serialization)와 Inspector 노출 규칙을 안다.
- Unity 스크립트 작성 시 흔한 실수와 성능 함정을 피할 수 있다.

1.2. 출발점: 일반 C#으로 게임을 만들 때 우리가 직접 한 일들

콘솔 또는 GUI 응용 프로그램(예: Windows Forms, WPF)으로 간단한 2D 게임을 직접 만든다고 상상해 보자. 다음을 모두 우리가 직접 작성해야 한다.

⚙️ 일반 C# 게임의 일반적 골격

```
Main()                                // 진입점
|
v
GameWindow / GameForm
|--- Timer (예: 16ms 마다 OnUpdate 호출)
|--- OnUpdate() : 위치 갱신, 충돌 검사
|--- OnDraw()   : Graphics 객체로 직접 그리기
|--- OnKeyDown() : 입력 추적
```

이 골격에는 다음 코드가 모두 “우리 손으로” 들어간다.

- 진입점 Main 작성 — 어디서 시작하고, 무엇을 떠올 것인가.
- 타이머/루프 직접 설치 — 16ms 간격으로 무엇을 호출할 것인가.
- 화면 갱신 요청 — “다시 그려라”라고 명시적으로 호출.
- 직접 그리기 — Graphics.FillRectangle, DrawImage 등을 매 프레임.
- 입력 추적 — 현재 어떤 키가 눌러 있는지 직접 관리.
- 수동 충돌 판정 — Math.Abs, 좌표 비교.

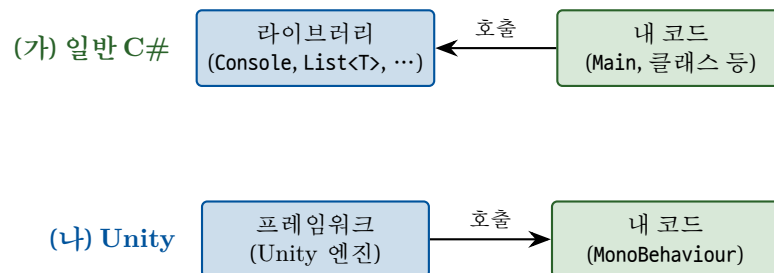
🔧 Unity의 출발점

Unity에서는 위 골격이 이미 만들어져 있다. Main을 작성할 필요도, 타이머를 만들 필요도, 화면 갱신을 호출할 필요도 없다. 대신 우리는 MonoBehaviour를 상속한 클래스를 작성하고 Update() 메서드만 채워 넣으면 된다. 나머지는 Unity 엔진이 알아서 호출해 준다.

1.3. 프레임워크와 라이브러리의 차이

🎬 할리우드 원칙

일반 C# 프로그램은 우리가 라이브러리를 호출한다 (→ `Console.WriteLine`).
 Unity 스크립트는 Unity 프레임워크가 우리를 호출한다 (→ `Update()`).
 이 “역전된 호출 관계”를 **IoC(Inversion of Control)** 또는 **할리우드 원칙** (“Don’t call us, we’ll call you”)이라고 부른다. 프레임워크가 일정한 약속(예: “Update라는 이름의 메서드는 매 프레임 호출함”)을 정해 두면, 우리는 그 약속에 맞춰 메서드를 작성하기만 하면 된다.



2. Unity 스크립트의 기본 구조

2.1. 가장 기본적인 Unity 스크립트

Unity Editor에서 “Create → MonoBehaviour Script”로 새 스크립트를 만들면 다음과 같은 코드가 자동으로 생성된다.

```

1 using UnityEngine;
2
3 public class HelloUnity : MonoBehaviour
4 {
5     void Start()
6     {
7         Debug.Log("Hello, Unity!");
8     }
9
10    void Update()
11    {
12        // Called every frame
13    }
14 }
  
```

이 짧은 코드 안에 **Unity 스크립트의 모든 핵심**이 들어 있다. 한 줄씩 살펴본다.

🔑 한 줄씩 분해

- `using UnityEngine;` — Unity의 모든 기본 클래스를 담은 네임스페이스. `MonoBehaviour`, `GameObject`, `Transform`, `Debug` 등이 모두 여기에 있다.
- `public class HelloUnity` — 클래스 이름은 파일 이름과 반드시 같아야 한다 (`HelloUnity.cs`). 다르면 Unity가 컴포넌트로 인식하지 못한다.
- `: MonoBehaviour` — Unity가 인식하는 “스크립트 컴포넌트”가 되려면 반드시 이 클래스를 상속해야 한다.
- `void Start()` — 게임 오브젝트가 활성화된 후 단 한 번 호출된다.
- `void Update()` — 매 프레임마다 Unity가 자동으로 호출한다.

- `Debug.Log(...)` — Unity의 Console 창에 메시지를 출력한다 (`Console.WriteLine`의 Unity 버전).

2.2. MonoBehaviour란 정확히 무엇인가?

`MonoBehaviour`는 Unity가 미리 만들어 둔 기반(base) 클래스이다. 이 클래스는 다음 두 가지를 가지고 있다:

1. 미리 정의된 “빈” 메서드들 — `Awake`, `Start`, `Update`, `OnCollisionEnter` 등 약 30개. 기본 구현은 “아무 것도 안 함”이다.
2. 유용한 속성들 — `transform`, `gameObject`, `enabled`, `name` 등.

우리가 `class HelloUnity : MonoBehaviour`라고 쓰면, 이 약속된 메서드 중 필요한 것만 골라서 재정의하는 셈이다.

⚠ 주의: override 키워드를 쓰지 않는다

일반 C#에서는 부모의 `virtual` 메서드를 자식에서 `override`로 표시해야 한다. Unity의 생명 주기 메서드는 `override`를 쓰지 않는다.

이는 Unity가 메서드 이름만 보고 리플렉션으로 찾아 호출하는 방식이기 때문이다. 메서드 이름의 철자가 틀리면 그냥 호출되지 않을 뿐 컴파일 에러는 발생하지 않으니 주의하라 (`Updaet`, `start`, `Awak` 같은 실수가 디버깅을 어렵게 만든다).

2.3. 일반 C# 프로그램과의 차이점

	일반 C# 프로그램	Unity 스크립트
진입점	<code>Main()</code> 메서드	없음 (Unity 엔진이 호출)
실행 단위	<code>.exe</code> 파일	<code>GameObject</code> 에 붙는 컴포넌트
객체 생성	<code>new MyClass()</code>	<code>AddComponent<MyClass>()</code>
필수 상속	없음	<code>MonoBehaviour</code>
파일 이름 규칙	자유	클래스 이름과 동일해야 함
프로그램 종료	<code>Main</code> 끝나면 종료	사용자가 정지 버튼 누를 때까지

i “Main이 없다”는 게 무슨 뜻인가?

일반 C# 프로그램에서 `Main`은 “프로그램의 시작점”이었다. Unity에서는 `Main`을 작성하지 않는다. Unity Editor에서 재생(Play) 버튼을 누르면 Unity 엔진이 다음 일을 자동으로 한다:

1. 현재 **Scene**에 있는 모든 `GameObject`를 찾는다.
2. 각 `GameObject`에 붙어 있는 `MonoBehaviour` 스크립트를 찾는다.
3. 각 스크립트의 `Awake` → `OnEnable` → `Start`를 차례로 호출.
4. 매 프레임마다 모든 활성 스크립트의 `Update`를 호출.

즉, Unity 엔진 자체가 거대한 `Main`이라고 생각하면 된다. 우리는 그 안에서 “특정 시점에 실행되어야 할 코드”만 약속된 이름의 메서드(`Start`, `Update` 등)에 작성한다.

2.4. Scene(씬)이란?

Scene은 “한 화면을 구성하는 GameObject들의 모음”이다. 하나의 게임은 보통 여러 개의 Scene으로 구성된다:

- TitleScene — 타이틀 화면 (로고, 시작 버튼)
- StageScene — 게임 플레이 화면 (플레이어, 적, UI)
- EndingScene — 엔딩 화면

Scene은 파일로 저장된다 (.unity 확장자). 저장된 Scene을 SceneManager.LoadScene으로 불러오면 이전 Scene의 GameObject들이 사라지고 새 Scene의 GameObject들이 생성된다.

⇄ 일반 GUI 프로그램과의 비교

일반 GUI 프로그램에서 “타이틀 화면 → 게임 화면”을 구현하려면 같은 윈도우 안에서 화면을 “갈아 끼우는” 코드를 직접 짜야 한다. Unity는 **Scene** 단위로 코드를 분리할 수 있어 자연스럽게 화면별 책임이 나뉜다.

3. MonoBehaviour의 생명 주기

Unity 스크립트는 시간 순서대로 호출되는 여러 메서드를 가진다. 이를 생명 주기(Lifecycle) 또는 이벤트 함수(Event Functions)라고 한다.

3.1. 핵심 생명 주기 메서드

메서드	호출 시점
Awake()	오브젝트가 생성될 때 한 번 (비활성 상태에서도 호출)
OnEnable()	오브젝트가 활성화될 때마다 호출
Start()	첫 Update 직전, 단 한 번 호출
Update()	매 프레임 호출 (보통 초당 30~120회)
FixedUpdate()	고정 시간 간격(기본 0.02초)마다 호출. 물리 처리용
LateUpdate()	모든 Update가 끝난 후 호출 (카메라 추적용)
OnDisable()	오브젝트가 비활성화될 때 호출
OnDestroy()	오브젝트가 삭제되기 직전 호출

3.2. Awake와 Start의 차이

학생들이 가장 자주 헷갈리는 부분이다. 둘 다 “초기화”에 사용되지만 호출 시점이 다르다.

```

1 using UnityEngine;
2
3 public class LifecycleDemo : MonoBehaviour
4 {
5     void Awake()
6     {
7         // Called immediately when this object is created.
8         // Use for: initializing this object's own state.
9         Debug.Log("Awake");
10    }
11
12    void Start()
13    {
14        // Called just before the first Update.

```

```

15     // Use for: accessing OTHER objects (they all have Awake'd by now).
16     Debug.Log("Start");
17 }
18
19 void Update()
20 {
21     // Called every frame.
22 }
23 }

```

❓ 왜 두 개로 나뉘어 있는가?

Scene에 A, B, C 세 오브젝트가 있다고 하자. 호출 순서는 다음과 같다:

1. A.Awake() → B.Awake() → C.Awake() — 이 시점에 모든 오브젝트의 “기본 준비”가 끝남.
2. A.Start() → B.Start() → C.Start() — 이 시점에 다른 오브젝트가 이미 Awake되어 있음이 보장됨.
3. A.Update() → B.Update() → C.Update() → ...

규칙

- Awake: 자기 자신의 변수 초기화, 같은 GameObject의 컴포넌트 캐싱
- Start: 다른 오브젝트와 상호작용해야 하는 초기화

3.3. Update와 FixedUpdate의 차이

	Update	FixedUpdate
호출 간격	프레임마다 (불규칙)	고정 (기본 0.02초, 50회/초)
용도	입력, 일반 로직, 비물리 이동	Rigidbody 등 물리 연산
시간 변수	Time.deltaTime	Time.fixedDeltaTime

✓ 언제 어느 것을 써야 하나?

- 키보드 입력 검사 → Update
(입력은 프레임 단위로 받는 것이 자연스럽다)
- transform.position을 직접 바꾸는 이동 → Update
- Rigidbody.AddForce, Rigidbody.velocity 변경 → FixedUpdate
(물리 엔진이 고정 시간 간격으로 동작하기 때문)
- 카메라가 캐릭터를 따라가는 코드 → LateUpdate
(캐릭터의 Update가 모두 끝난 후 카메라가 따라가야 끊김 없음)

3.4. Rigidbody2D와 물리 엔진

Rigidbody(2D는 Rigidbody2D)는 “이 오브젝트는 물리 법칙의 영향을 받는다”라고 알리는 컴포넌트이다. 이 컴포넌트가 붙은 오브젝트에는 다음이 자동으로 적용된다:

- 중력 — 아래로 자유 낙하
- 관성 — 한번 움직이면 마찰이 없는 한 계속 움직임

- 충돌 반응 — 다른 Collider에 부딪히면 튕겨 나감

⇄ 일반 C#과의 비교

일반 C#에서 “공이 벽에서 튀어 나오기”를 직접 코딩한다면

```
if (x < 0 || x > maxX) dx = -dx;
```

같은 검사를 우리가 매 프레임 짜야 한다.

Unity에서는 Rigidbody2D와 Collider2D, 그리고 “Bouncy” Physics Material 2D만 설정하면 공식이 필요 없이 알아서 튕긴다.

왜 FixedUpdate에서 다뤄야 하나? 물리 엔진은 정확한 시뮬레이션을 위해 고정된 시간 간격으로 계산을 해야 한다. 프레임 속도(가변)에 맞춰 물리를 계산하면, 빠른 컴퓨터에서는 공이 더 빨리 떨어지고 느린 컴퓨터에서는 천천히 떨어지는 이상한 현상이 생긴다. FixedUpdate는 항상 0.02초 간격으로 호출되므로 어떤 기기에서도 동일한 물리 결과가 나온다.

3.5. Time.deltaTime의 의미

Update는 프레임마다 호출되지만, 프레임 간격이 일정하지 않다. 어떤 프레임은 1/60초, 어떤 프레임은 1/30초가 걸릴 수 있다.

```
1 void Update()
2 {
3     // Wrong: speed depends on frame rate
4     transform.position += new Vector3(0.1f, 0, 0);
5
6     // Correct: speed independent of frame rate (5 units per second)
7     transform.position += new Vector3(5f * Time.deltaTime, 0, 0);
8 }
```

i Time.deltaTime이란?

Time.deltaTime은 “이전 프레임으로부터 지금까지 경과한 시간(초)”이다.

- 60fps → Time.deltaTime ≈ 0.0167초
- 30fps → Time.deltaTime ≈ 0.0333초

이동량 = 속도 × 시간 이므로, 속도 * Time.deltaTime을 더하면 프레임 속도와 무관하게 일정한 속도로 움직인다.

4. 컴포넌트 기반 설계

4.1. GameObject와 Component

Unity는 상속이 아니라 조합(Composition)으로 객체를 만든다.

⇄ 일반 C# vs Unity

일반 C# 방식 (상속/직접 작성):

```
class Player {
    public float X, Y;
    void Update(...) { ... }
    void Draw(...) { ... }
}
```

Unity 방식 (컴포넌트 조합):

```
GameObject "Player"
  +-- Transform      (position, rotation, scale)
  +-- SpriteRenderer (image)
  +-- Rigidbody2D    (physics)
  +-- BoxCollider2D  (collision shape)
  +-- PlayerController (our script)
```

GameObject 자체는 빈 컨테이너이다. “무엇을 할 수 있는가”는 거기에 붙어 있는 Component들이 결정한다. 이 사고방식은 처음에는 어색하지만, 익숙해지면 매우 강력하다.

4.2. 상속(Inheritance) vs 조합(Composition)

객체에 기능을 부여하는 두 가지 방식이 있다.

상속 방식 (전통 OOP) “Player는 Character이다”라는 is-a 관계.

```
class Character { ... }
class Movable : Character { ... }
class FlyingCharacter : Movable { ... }
class Player : FlyingCharacter { ... }
```

새 기능을 추가하려면 새 자식 클래스를 만들어야 한다. “걸을 수 있는 적”이 필요하면 또 새 클래스, “날 수 있는 아이템”이 필요하면 또 새 클래스 — 계층이 점점 깊어지고 복잡해진다 (이른바 “상속 지옥”).

조합 방식 (Unity) “Player는 Mover를 가진다”라는 has-a 관계.

```
GameObject "Player"
  + Mover          // movement ability
  + Shooter        // shooting ability
  + HealthComponent // hp tracking
```

“걸을 수 있는 적”이 필요하면 Enemy GameObject에 Mover만 붙이면 된다. 새 클래스가 필요 없다. 기능들이 레고 블록처럼 조립된다.

★ 한 줄 요약

상속이 “무엇이다(what it is)”라면, 조합은 “무엇을 가진다(what it has)”이다. Unity는 의도적으로 후자를 선택했다.

컴포넌트 모델의 장점

- 재사용성: Rigidbody2D는 플레이어든 적이든 누구든 붙일 수 있다.
- 유연성: 게임 도중에 컴포넌트를 추가/제거해서 동작을 바꿀 수 있다.
- 단순한 클래스: 각 컴포넌트는 하나의 일만 한다. “상속 지옥”(deep inheritance)이 발생하지 않는다.

4.3. GetComponent로 컴포넌트 접근

같은 GameObject에 붙은 다른 컴포넌트를 가져오려면 GetComponent<T>()를 사용한다.

```

1 using UnityEngine;
2
3 public class PlayerController : MonoBehaviour
4 {
5     Rigidbody2D rb;
6     SpriteRenderer sr;
7
8     void Awake()
9     {
10         // Cache references to other components on the same GameObject
11         rb = GetComponent<Rigidbody2D>();
12         sr = GetComponent<SpriteRenderer>();
13     }
14
15     void Update()
16     {
17         // Use the cached references
18         if (Input.GetKeyDown(KeyCode.Space))
19         {
20             rb.velocity = new Vector2(0, 5);
21             sr.color = Color.red;
22         }
23     }
24 }

```

⚠ 왜 Awake에서 캐싱하는가?

GetComponent<T>()는 매번 호출될 때마다 컴포넌트 목록을 검색한다. Update에서 매 프레임 호출하면 성능에 손해이다.

```

// BAD: searches every frame (60+ times per second)
void Update() {
    GetComponent<Rigidbody2D>().velocity = ...;
}

// GOOD: search once, reuse the reference
Rigidbody2D rb;
void Awake() { rb = GetComponent<Rigidbody2D>(); }
void Update() { rb.velocity = ...; }

```

이 패턴은 Unity 스크립트의 거의 모든 곳에서 사용된다. “Update에서 GetComponent를 호출하지 마라”는 가장 기본적인 성능 규칙이다.

4.4. Transform: 모든 GameObject의 필수 컴포넌트

모든 GameObject는 Transform 컴포넌트를 반드시 하나 가진다. 이는 위치(position), 회전(rotation), 크기(scale)를 담고 있다.

```

1 public class Mover : MonoBehaviour
2 {
3     public float speed = 3f;
4
5     void Update()
6     {
7         // Move 3 units per second to the right
8         transform.position += new Vector3(speed * Time.deltaTime, 0, 0);
9
10        // Rotate 90 degrees per second around the Z axis

```

```

11     transform.Rotate(0, 0, 90f * Time.deltaTime);
12 }
13 }

```

💡 transform은 어디서 왔나?

MonoBehaviour는 내부적으로 transform 속성을 이미 가지고 있다. 별도로 GetComponent<Transform>()을 호출할 필요 없이 그냥 transform이라고 쓰면 된다. 이는 너무 자주 쓰이는 컴포넌트이기 때문에 Unity가 미리 노출시켜 둔 것이다. 비슷하게 gameObject 속성도 “이 컴포넌트가 붙어 있는 GameObject”를 가리킨다.

📦 Vector3란 무엇인가?

Vector3는 “3차원 공간의 한 점 또는 방향”을 나타내는 구조체이다. 세 개의 float 값(x, y, z)으로 이루어져 있다.

```

Vector3 p = new Vector3(2f, 5f, 0f);
Debug.Log(p.x);    // 2
Debug.Log(p.y);    // 5

// Vector arithmetic
Vector3 a = new Vector3(1, 0, 0);
Vector3 b = new Vector3(0, 1, 0);
Vector3 c = a + b;    // (1, 1, 0)
Vector3 d = a * 5f;   // (5, 0, 0)

// Useful constants
Vector3.zero;        // (0, 0, 0)
Vector3.up;          // (0, 1, 0)
Vector3.right;        // (1, 0, 0)

```

왜 x, y, z를 따로 두지 않고 Vector3로 묶나?

- 한 단위로 다루기 위해 — transform.position = new Vector3(...) 한 줄로 위치를 통째로 설정.
- 벡터 연산 지원 — +, -, * 같은 연산자가 의미 그대로 작동.

2D 게임에서는 z를 0으로 두거나 Vector2(x, y만 있음)를 쓴다.

5. Inspector와 직렬화

5.1. public 필드는 Inspector에 노출된다

Unity는 스크립트의 필드를 자동으로 Inspector에 표시한다. 이를 통해 코드를 수정하지 않고도 값을 바꿀 수 있다.

```

1 public class Mover : MonoBehaviour
2 {
3     public float speed = 3f;        // Shown in Inspector
4     public Color color = Color.red; // Shown in Inspector
5     public GameObject target;       // Shown in Inspector (drag-and-drop)
6
7     private int hiddenValue = 10;   // NOT shown in Inspector
8 }

```

이는 일반 C# 프로그램에서는 보지 못한 새로운 개념이다. “디자이너는 코드를 모르지만 값은 조정할 수 있어야 한다”는 게임 개발 현장의 요구가 반영된 것이다.

Inspector 노출 규칙

- public 필드 → 자동 노출 (단, 직렬화 가능한 타입이어야 함)
- private 필드 → 노출되지 않음
- [SerializeField] private 필드 → 노출됨
- public + [HideInInspector] → 숨겨짐
- 프로퍼티(public int X { get; set; }) → 노출되지 않음

직렬화 가능한 타입: 기본형(int, float, ...), string, Vector2/3, Color, GameObject, Transform, MonoBehaviour를 상속한 클래스 등.

5.2. 권장 패턴: [SerializeField] private

public 필드는 Inspector에 노출되지만, 동시에 다른 스크립트에서도 자유롭게 접근 가능하다. 이는 캡슐화에 좋지 않다. Unity 커뮤니티에서는 다음 패턴을 권장한다:

```
1 public class Player : MonoBehaviour
2 {
3     // Editable in Inspector, but NOT accessible from other scripts
4     [SerializeField] private float speed = 3f;
5     [SerializeField] private int maxHp = 100;
6
7     // Read-only access from outside
8     public int CurrentHp { get; private set; }
9 }
```

💡 [SerializeField] 한 줄로 두 마리 토끼

- Inspector에서 조정 가능: 디자이너가 값을 바꿀 수 있다.
- 외부에서 접근 불가: private이므로 다른 클래스가 함부로 못 건드린다.

이는 OOP의 캡슐화 원칙을 Unity 환경에 맞춘 것이다. “필요한 것만 외부에 공개한다”는 OOP 원칙은 Unity에서도 그대로 유효하다.

5.3. Inspector가 있다는 것의 의미

🔧 일반 C#과의 결정적 차이

일반 C# 프로그램에서 “플레이어 속도”를 0.5만큼 조정하려면 다음 사이클을 거쳐야 한다.
→ 코드 수정 → 재컴파일 → 실행 → 확인 → 다시 수정 ...

한 번 사이클에 수십 초가 걸린다.

Unity에서는 [SerializeField] float speed만 선언해 두면

→ Inspector에서 슬라이더 드래그 → 즉시 반영

심지어 플레이 중에도 값을 바꿔 가며 “느낌”을 잡을 수 있다. 이것이 게임 개발 생산성의 핵심 차이를 만든다.

6. Unity 스크립트의 주요 고려 사항

6.1. (1) MonoBehaviour는 new로 만들지 마라

일반 C# 클래스는 new MyClass()로 인스턴스를 만든다. MonoBehaviour는 다르다.

```
1 // WRONG: This compiles but Unity will not call Awake/Start/Update on it
2 PlayerController p = new PlayerController();
3
4 // CORRECT: attach to a GameObject
5 GameObject obj = new GameObject("Player");
6 PlayerController p = obj.AddComponent<PlayerController>();
7
8 // ALSO CORRECT: instantiate from a Prefab
9 GameObject p2 = Instantiate(playerPrefab);
```

⚠ 왜 new는 안 되는가?

MonoBehaviour는 “GameObject에 붙어서 동작하는 컴포넌트”이다. GameObject 없이 단독으로 존재하는 것은 의미가 없다.

new로 만들면 객체 자체는 생성되지만:

- Unity 엔진이 그 존재를 모른다 → Update가 호출되지 않는다.
- transform, gameObject 속성이 null이다.
- Destroy(this)를 호출해도 동작하지 않는다.

반드시 AddComponent<T>() 또는 Instantiate()를 사용해야 한다.

✂ AddComponent vs Instantiate vs Prefab

세 가지가 자주 같이 등장하지만 역할이 다르다.

AddComponent<T>() 이미 존재하는 GameObject에 컴포넌트 하나를 추가한다.

```
GameObject obj = new GameObject("Player");
obj.AddComponent<Rigidbody2D>(); // physics ability
obj.AddComponent<PlayerController>(); // our script
```

Instantiate(prefab) Prefab을 복제하여 Scene에 새 GameObject를 만든다. 이미 만들어 둔 “총알 프리팹”을 매번 복제해서 화면에 발사하는 식.

```
public GameObject bulletPrefab; // assigned in Inspector
GameObject b = Instantiate(bulletPrefab,
    transform.position,
    Quaternion.identity);
```

Prefab(프리팹)이란? “재사용 가능한 GameObject 템플릿”이다. 복잡한 GameObject(예: 모델 + 애니메이션 + 충돌체 + 스크립트가 모두 붙은 적 캐릭터)를 한 번 만들어 두고 .prefab 파일로 저장하면, 게임 도중 Instantiate로 무한히 복제 가능.

언제 어느 것을 쓰나?

- 단순한 일회성 객체 → new GameObject + AddComponent

- 자주 만들어지는 복잡한 객체(총알, 적, 아이템) → Prefab + Instantiate

6.2. (2) Update에서 무거운 작업을 하지 마라

Update는 초당 수십 수백 번 호출된다. 여기에 무거운 코드가 들어가면 게임 전체가 느려진다.

```

1 // BAD: searches the entire scene every frame
2 void Update()
3 {
4     GameObject enemy = GameObject.Find("Enemy");
5     enemy.transform.Translate(Vector3.left * Time.deltaTime);
6 }
7
8 // GOOD: find once, reuse
9 GameObject enemy;
10 void Start()
11 {
12     enemy = GameObject.Find("Enemy");
13 }
14 void Update()
15 {
16     enemy.transform.Translate(Vector3.left * Time.deltaTime);
17 }

```

⚠ Update에서 피해야 할 것들

- `GameObject.Find(...)` — Scene 전체 검색
- `FindObjectOfType<T>()` — 모든 오브젝트 순회
- `GetComponent<T>()` — 컴포넌트 목록 검색 (캐싱하라)
- `new SomeClass()` 반복 — 매 프레임 객체 생성 시 가비지 컬렉션 부담
- `Debug.Log(...)` 남발 — 의외로 비용이 크다. 디버깅 후 지워야 한다.

“찾는 것은 한 번, 사용하는 것은 매번”이 기본 원칙이다.

🗑 가비지 컬렉션(GC)이 게임에서 왜 문제인가?

C#은 사용하지 않는 객체의 메모리를 자동으로 정리해 준다. 이를 가비지 컬렉션(GC)이라고 한다. 편리한 기능이지만 게임에서는 골칫거리이다.

문제: 프레임 끊김 GC는 “메모리 청소”를 위해 프로그램을 잠시 멈춘다. 일반 프로그램에서는 무시할 수준이지만, 60fps 게임은 한 프레임에 16ms뿐이므로 GC가 5ms만 발생해도 “끊김(stutter)”이 눈에 띈다.

```

void Update()
{
    // BAD: creates new object every frame -> GC pressure
    var dir = new System.Collections.Generic.List<int>();
    /* ... */
}

```

Vector3는 구조체라 GC와 무관하지만, new List<>(), new string() 같은 참조 타입을 매 프레임 만드는 것은 GC를 자주 깨우게 한다.

해결: 미리 만들고 재사용

- 필드로 미리 객체를 만들어 둔다 (Awake/Start에서)
- 문자열 합치기는 string + 대신 StringBuilder 사용
- 자주 생성/파괴되는 오브젝트는 **Object Pooling**(객체 풀) 패턴 사용

이는 일반 C#에서는 거의 신경 안 쓰는 부분이지만, Unity에서는 기본 상식이다.

6.3. (3) Unity의 null은 일반 C#의 null과 다르다

UnityEngine.Object(즉 GameObject, Transform 등)에 대한 == null 비교는 실제로는 Unity가 오버라이드한 비교 연산이 수행된다.

```

1  GameObject obj = ...;
2  Destroy(obj); // Object is marked for destruction
3
4  // One frame later:
5  if (obj == null)           // TRUE -- Unity reports it as null
6      Debug.Log("destroyed");
7
8  // But the C# reference is technically not null!
9  object asObj = obj;
10 if (asObj == null)         // FALSE -- the reference still exists
11     ...

```

💡 “가짜 null”(Fake Null) 현상

Destroy()로 삭제된 Unity 오브젝트는 C# 입장에서는 여전히 참조가 살아 있지만, Unity가 operator==를 오버로드해서 == null이면 true를 반환하게 만들었다.

실용적 규칙 Unity 오브젝트는 항상 == null로 비교하라. ?. 같은 null 조건 연산자나 ?? null 합치기 연산자는 이 “가짜 null”을 인식하지 못하므로 의도와 다르게 동작할 수 있다.

```

// Subtle bug
target?.SomeMethod(); // Calls method on a destroyed object!

// Safe
if (target != null) target.SomeMethod();

```

6.4. (4) Coroutine: 시간에 따라 펼쳐지는 작업

“2초 기다린 후 폭발”같은 시간 흐름이 있는 작업을 Update에서 처리하려면 매우 복잡해진다. Unity는 이를 위해 코루틴(Coroutine)을 제공한다.

```

1  using System.Collections;
2  using UnityEngine;
3
4  public class Bomb : MonoBehaviour
5  {

```



```

6  void Start()
7  {
8      StartCoroutine(ExplodeAfterDelay());
9  }
10
11  IEnumerator ExplodeAfterDelay()
12  {
13      Debug.Log("Bomb planted");
14      yield return new WaitForSeconds(2f);
15      Debug.Log("BOOM!");
16      Destroy(gameObject);
17  }
18 }

```

📌 코루틴 한 줄 요약

IEnumerator를 반환하는 메서드 안에서 yield return으로 “잠시 멈춰라”라고 표시할 수 있다.

- yield return null; — 다음 프레임까지 대기
- yield return new WaitForSeconds(2f); — 2초 대기
- yield return new WaitUntil(() => isReady); — 조건이 참이 될 때까지 대기

이는 “비동기 코드를 마치 동기처럼 쓰게 해 주는” Unity의 기능이다. 일반 C#의 async/await와 비슷한 개념이지만, Unity의 게임 루프와 잘 어울리도록 설계되어 있다.

❓ 일반 C#이라면 어떻게?

“2초 후 폭발”을 일반 C#으로 짜려면

(a) Thread.Sleep(2000) → 프로그램이 멈춰서 입력도 못 받음

(b) Timer 콜백 → 콜백 지옥, 상태 관리 복잡

(c) async/await → 가능하지만 게임 루프와 잘 안 맞음

코루틴은 (1) 메인 스레드에서 안전하게 실행되고, (2) 메서드 안의 지역 변수가 자동 보존되며, (3) 게임 루프와 자연스럽게 동기화된다.

6.5. (5) 직접 그리지 않는다

일반 GUI 프로그램에서는 “Paint 이벤트”에서 Graphics 객체로 직접 도형을 그렸다. Unity에서는 그렇지 않다.

- 이미지는 SpriteRenderer 컴포넌트가 그린다.
- 3D 모델은 MeshRenderer가 그린다.
- UI 텍스트는 TextMeshProUGUI가 그린다.

우리 스크립트는 무엇을 그릴지 결정만 한다(위치 변경, 색상 변경, sprite 교체 등). 실제 그리기는 Unity의 렌더링 시스템이 담당한다.

🔧 예: 색상 바꾸기

```

public class ColorChanger : MonoBehaviour
{
    SpriteRenderer sr;
}

```

```
void Awake() { sr = GetComponent<SpriteRenderer>(); }

void Update()
{
    if (Input.GetKeyDown(KeyCode.R)) sr.color = Color.red;
    if (Input.GetKeyDown(KeyCode.G)) sr.color = Color.green;
}
}
```

우리는 `sr.color = ...`로 “빨간색으로 그려”라고 지시만 한다. 실제 픽셀을 칠하는 것은 Unity의 렌더러이다.

★ 렌더러(Renderer) 컴포넌트 종류

컴포넌트	용도
SpriteRenderer	2D 이미지(스프라이트) 표시
MeshRenderer	3D 메쉬(모델) 표시
TextMeshProUGUI	UI 텍스트 (점수, HP 등)
LineRenderer	선/궤적 (총알 자국, 레이저)
ParticleSystem	파티클 효과 (불꽃, 연기)

GameObject에 위 컴포넌트를 붙이고 sprite/메시 등을 지정해 두면 끝. 이후로는 어떤 그리기 코드도 작성하지 않아도 매 프레임 자동으로 그려진다. 스크립트는 `transform.position`이나 `sr.color`를 바꿔서 “어디에/어떻게” 그릴지만 결정한다.

7. 입력과 충돌 처리

7.1. 한눈에 보는 비교: 일반 C#/GUI vs Unity

田 같은 일을 어떻게 하나

	일반 C#/GUI 프로그램	Unity
진입점	Main / Application.Run	없음 (엔진이 자동)
게임 루프	Timer/while 직접 작성	Update 자동 호출
프레임 보정	직접 시간 측정	Time.deltaTime
화면 갱신	Invalidate + Paint	자동 렌더링
그리기	Graphics.FillRectangle 등	SpriteRenderer
객체 모델	클래스 직접 작성	GameObject + Component
입력 처리	KeyDown 이벤트 핸들러	Input.GetKey 폴링
충돌 판정	Math.Abs(...) < N 직접	Collider + 콜백
값 조정	코드 수정 후 재컴파일	Inspector에서 즉시

7.2. Input.GetKey 세 가지 변형

일반 GUI 프로그래밍에서 “현재 어떤 키가 눌러 있는지”를 추적하려면 KeyDown/KeyUp 이벤트를 HashSet 같은 곳에 모아 직접 관리해야 했다. Unity에서는 그 작업이 한 줄로 끝난다. 다만 세 가지 변형이 있고, 차이를 정확히 알아야 한다.

메서드	언제 true를 반환하는가
Input.GetKey(K)	키가 눌러 있는 동안 매 프레임 true (이동에 사용)
Input.GetKeyDown(K)	키를 누른 그 프레임에 한 번만 true (점프, 발사에 사용)
Input.GetKeyUp(K)	키를 떼는 그 프레임에 한 번만 true (차징 해제 등)

```
void Update()
{
    // Continuous movement: while held
    if (Input.GetKey(KeyCode.LeftArrow))
        transform.Translate(-5 * Time.deltaTime, 0, 0);

    // One-shot action: only the moment of press
    if (Input.GetKeyDown(KeyCode.Space))
        Shoot();
}
```

⚠️ 흔한 실수

점프에 GetKey를 쓰면, Space 키를 길게 누르고 있는 동안 매 프레임 점프 코드가 실행되어 캐릭터가 미친 듯이 튀어 오른다. “한 번 누름 = 한 번 동작”인 행동에는 반드시 GetKeyDown을 써야 한다.

7.3. Collider와 충돌 콜백

일반 C#에서는 충돌을 직접 계산했다 ($\text{Math.Abs}(b.X - X) < 20$). Unity에서는 GameObject에 Collider(2D는 Collider2D)를 붙이면 충돌 검사를 엔진이 대신 해 주고, 결과를 콜백 메서드로 알려 준다.

Collider 종류

- BoxCollider2D — 사각형 (간단, 빠름)
- CircleCollider2D — 원형
- PolygonCollider2D — 임의의 다각형 (복잡, 느림)

```
void OnCollisionEnter2D(Collision2D c) {
    // when a Rigidbody Collider hits another Collider
    Debug.Log("Hit: " + c.gameObject.name);
}

void OnTriggerEnter2D(Collider2D c) {
    // when entering a "trigger" zone (no physics push)
    Debug.Log("Entered: " + c.gameObject.name);
}
```

🔍 Collision vs Trigger의 차이

- Collision — “실제로 부딪히는” 충돌. 물리적으로 튕긴다 (벽, 바닥).
- Trigger — “통과는 되지만 알림은 받는” 충돌. 안 튕긴다 (아이템 획득 영역, 트리거 게이트).

Collider의 **Is Trigger** 체크박스를 켜면 Trigger 모드가 된다.

7.4. Unity가 대신 해 주는 것들

충돌이 일어나면 자동으로 호출되는 메서드

★ 요약

Unity에서 “코드가 짧다”고 느껴지는 이유는, 우리가 작성하지 않아도 되는 부분이 매우 많기 때문이다:

- 진입점(Main), 메시지 루프, 창 생성
- 프레임 타이머, 더블 버퍼링, 화면 갱신
- 키보드/마우스/터치 입력의 저수준 처리
- 물리 시뮬레이션 (중력, 충돌, 마찰)
- 텍스처/메시 로딩, 셰이더, 렌더링 파이프라인
- 오디오 재생, 네트워크, 파일 시스템 추상화

대신 우리는 “내 게임의 규칙”만 쓰면 된다. 이것이 게임 엔진의 가치이고, Unity 스크립트가 일반 C#과 다른 모양을 가지는 이유이다.

8. 실전 예제: 충돌하면 합쳐지는 공들

지금까지 배운 개념을 한 번에 적용하는 작은 게임을 만든다. 화면에 여러 개의 공이 떠다니다가 벽에 부딪히면 튕기고, 공끼리 부딪히면 하나로 합쳐지는 시뮬레이션이다.

이 예제 하나에 다음 개념이 모두 들어 있다:

- MonoBehaviour, Awake/Start 초기화
- Rigidbody2D + Collider2D로 자동 물리/충돌
- OnCollisionEnter2D 충돌 콜백
- Prefab + Instantiate로 다수 객체 생성
- [SerializeField]로 Inspector 노출
- Destroy로 오브젝트 제거

8.1. 게임 컨셉과 물리 규칙

⚙ 동작 명세

1. 시작 시 화면에 **N개의 공**이 무작위 위치에 생성된다.
2. 각 공은 무작위 방향으로 일정 속도로 움직인다.
3. 공이 벽에 부딪히면 튕긴다 (운동 에너지 보존).
4. 공 두 개가 서로 충돌하면 하나로 합쳐진다.
 - 합쳐진 공의 면적은 두 공의 면적의 합 ($r_{new} = \sqrt{r_1^2 + r_2^2}$)
 - 합쳐진 공의 질량은 두 공의 질량의 합

- 합쳐진 공의 속도는 운동량 보존 ($\vec{v}_{new} = \frac{m_1\vec{v}_1 + m_2\vec{v}_2}{m_1 + m_2}$)
- 합쳐진 공의 위치는 질량 중심 위치

💡 왜 면적을 보존하는가?

2D에서 “공의 면적”은 3D의 “부피”에 해당한다. 물리적으로 두 물체가 합쳐질 때 부피(면적)가 보존된다고 보면 자연스럽다.

원의 면적은 πr^2 이므로, $\pi r_1^2 + \pi r_2^2 = \pi r_{new}^2$ 에서 $r_{new} = \sqrt{r_1^2 + r_2^2}$ 이다.

만약 단순히 $r_{new} = r_1 + r_2$ 로 하면 “공이 너무 빨리 커지는” 비현실적인 결과가 나온다.

8.2. Hierarchy 설계

Unity Editor의 **Hierarchy** 창에 만들 GameObject 구조는 다음과 같다.

🏠 Scene Hierarchy

Main Camera	(Unity가 기본 제공)
GameManager	(빈 GameObject)
+--- BallSpawner.cs	(스크립트 컴포넌트)
Walls	(빈 GameObject -- 그룹 정리용)
+--- WallTop	(BoxCollider2D + SpriteRenderer)
+--- WallBottom	
+--- WallLeft	
+--- WallRight	

공(Ball)은 Hierarchy에 미리 두지 않고 **Prefab**으로만 만들어 둔 다음 BallSpawner가 게임 시작 시에 Instantiate로 생성한다.

📁 빈 GameObject로 그룹을 만드는 이유

Walls라는 빈 GameObject 아래에 4개의 벽을 두면:

- Hierarchy가 시각적으로 정리된다.
- Walls.SetActive(false)로 4개 벽을 한 번에 비활성화할 수 있다.
- Walls.transform.position을 옮기면 4개 벽이 함께 이동한다.

이는 “관련 있는 것끼리 묶어 둔다”는 OOP 원칙을 Hierarchy에 적용한 것이다.

8.3. Prefab 설계: BallPrefab

공 하나에 필요한 컴포넌트는 다음과 같다.

컴포넌트	역할
Transform	위치/크기 (필수, 자동 포함)
SpriteRenderer	원 모양 이미지 표시
CircleCollider2D	원형 충돌 영역
Rigidbody2D	물리 (관성, 충돌 응답)
Ball.cs	우리가 작성할 스크립트

Rigidbody2D는 다음과 같이 설정한다:

- Gravity Scale = 0 — 중력 없음 (위에서 떨어지지 않게)

- Linear Drag = 0 — 공기 저항 없음 (계속 움직이게)
- Collision Detection = Continuous — 빠른 공 통과 방지

또한 **Physics Material 2D**를 만들어 **CircleCollider2D**에 할당한다:

- Friction = 0 — 마찰 없음
- Bounciness = 1 — 완전 탄성 충돌 (속도 보존)

✔ 우리가 작성하는 코드는 얼마나 적은가?

공이 “무작위로 움직이고, 벽에 튕기고, 다른 공과 부딪히는” 동작 중 “무작위 초기 속도 부여”와 “공-공 충돌 시 합치기”만 우리가 코딩한다. 나머지(움직임, 벽 충돌, 튕김 각도)는 **Rigidbody2D + Collider2D + Physics Material**이 자동으로 한다.

같은 게임을 일반 C#/GUI로 짤다면 dx, dy로 직접 속도를 관리하고 if (x < 0) dx = -dx;로 반사를 직접 코딩해야 한다.

8.4. 스크립트 설계: 역할 분리

스크립트는 두 개로 나눈다.

📁 두 개의 스크립트

- Ball.cs — 개별 공의 동작
 - 시작 시 무작위 방향으로 발사
 - 다른 공과 충돌하면 합치기
- BallSpawner.cs — 전체 게임 관리
 - 시작 시 N개의 공을 생성

이는 “한 클래스는 하나의 일” 원칙을 따른 것이다. BallSpawner는 “공을 만든다”는 일만, Ball은 “공이 어떻게 행동하는가”만 안다.

⚠ 충돌 시 “누가 합치는가” 문제

공 A와 공 B가 충돌하면 OnCollisionEnter2D가 두 번 호출된다 (A에서 B를 만남, B에서 A를 만남). 양쪽이 모두 합치기 코드를 실행하면 Destroy가 두 번 일어나거나 합치기가 두 번 실행되는 버그가 생긴다.

해결: ID 비교로 “더 작은 쪽이 처리” GameObject.GetInstanceID()는 모든 오브젝트마다 고유한 정수이다. “ID가 더 작은 쪽”만 합치기 코드를 실행하면 정확히 한 번만 처리된다.

8.5. Ball.cs 구현

```

1 using UnityEngine;
2
3 public class Ball : MonoBehaviour
4 {
5     [SerializeField] private float initialSpeed = 3f;
6
7     private Rigidbody2D rb;
8
9     void Awake()
```

```
10 {
11     rb = GetComponent<Rigidbody2D>();
12 }
13
14 void Start()
15 {
16     // Random initial direction
17     float angle = Random.Range(0f, 360f) * Mathf.Deg2Rad;
18     Vector2 dir = new Vector2(Mathf.Cos(angle), Mathf.Sin(angle));
19     rb.velocity = dir * initialSpeed;
20 }
21
22 void OnCollisionEnter2D(Collision2D c)
23 {
24     // Try to find a Ball component on the other object
25     Ball other = c.gameObject.GetComponent<Ball>();
26
27     // If null, it was a wall - let physics handle it (bounce)
28     if (other == null) return;
29
30     // Process merge only once: the ball with smaller ID handles it
31     if (GetInstanceID() > other.GetInstanceID()) return;
32
33     Merge(other);
34 }
35
36 void Merge(Ball other)
37 {
38     // Current radii (sprite is 1x1 unit by default)
39     float r1 = transform.localScale.x * 0.5f;
40     float r2 = other.transform.localScale.x * 0.5f;
41
42     // Conservation of area:  $r_{\text{new}}^2 = r_1^2 + r_2^2$ 
43     float rNew = Mathf.Sqrt(r1 * r1 + r2 * r2);
44
45     // Conservation of mass
46     float m1 = rb.mass;
47     float m2 = other.rb.mass;
48     float mNew = m1 + m2;
49
50     // Conservation of momentum
51     Vector2 vNew = (m1 * rb.velocity + m2 * other.rb.velocity) / mNew;
52
53     // Center of mass position
54     Vector2 p1 = transform.position;
55     Vector2 p2 = other.transform.position;
56     Vector2 pNew = (m1 * p1 + m2 * p2) / mNew;
57
58     // Apply to this ball
59     transform.position = pNew;
60     transform.localScale = Vector3.one * (rNew * 2f);
61     rb.mass = mNew;
62     rb.velocity = vNew;
63
64     // Remove the other ball
65     Destroy(other.gameObject);
66 }
67 }
```

🔍 코드 한 부분씩 살펴보기

- [SerializeField] private float initialSpeed — Inspector에서 초기 속도를 조정할 수 있게 노출.
- rb = GetComponent<Rigidbody2D>() 를 Awake에서 — 한 번만 검색해서 캐싱 (성능 규칙).
- Start에서 무작위 방향 — 모든 공의 Awake가 끝난 후 호출되므로 모든 공이 동시에 동작 시작.
- c.gameObject.GetComponent<Ball>() — 부딪힌 상대가 공인지 벽인지 판별. 벽에는 Ball.cs 가 없으므로 null이 반환되어 “벽 처리”로 분기.
- GetInstanceID() > other.GetInstanceID()로 한 번만 처리 — “양쪽이 모두 합치는 버그” 방지.
- Destroy(other.gameObject) — 다른 공을 컴포넌트가 아니라 **GameObject** 단위로 제거. Destroy(other)만 하면 컴포넌트만 사라지고 GameObject는 남는다.

8.6. BallSpawner.cs 구현

```

1 using UnityEngine;
2
3 public class BallSpawner : MonoBehaviour
4 {
5     [SerializeField] private GameObject ballPrefab;
6     [SerializeField] private int count = 10;
7     [SerializeField] private Vector2 areaMin = new Vector2(-7, -4);
8     [SerializeField] private Vector2 areaMax = new Vector2(7, 4);
9
10    void Start()
11    {
12        for (int i = 0; i < count; i++)
13        {
14            float x = Random.Range(areaMin.x, areaMax.x);
15            float y = Random.Range(areaMin.y, areaMax.y);
16            Vector3 pos = new Vector3(x, y, 0);
17
18            Instantiate(ballPrefab, pos, Quaternion.identity);
19        }
20    }
21 }

```

🔧 ballPrefab은 어떻게 채워지는가?

[SerializeField] private GameObject ballPrefab; 한 줄을 쓰면 Inspector에 Ball Prefab이라는 빈 슬롯이 생긴다. Project 창에서 만든 BallPrefab을 그 슬롯으로 드래그 앤 드롭하면 연결된다.

왜 이렇게 하나? 대안은 코드에서 Resources.Load("BallPrefab")로 직접 찾는 방법이지만:

- 문자열 오타에 취약 ("Ballprefab" 같은 실수)
- 프리팹 이름이 바뀌면 모든 코드를 고쳐야 함

Inspector 슬롯에 드래그하면 **Unity**가 참조 자체를 저장하므로 이름이 바뀌어도 자동으로

따라간다.

8.7. Unity Editor에서 적용하기 (단계별)

⚙ Step-by-Step 셋업

Step 1: 새 2D 프로젝트 생성

Unity Hub에서 “New Project” → “2D Core” 선택.

Step 2: 벽 만들기

- Hierarchy에서 “Create Empty”로 Walls 생성.
- Walls 아래에 “2D Object > Sprites > Square” 4개 생성.
- 각각 WallTop, WallBottom, WallLeft, WallRight로 이름 변경.
- Scale과 Position을 조정해 화면 가장자리에 배치.
- 각 벽에 Add Component → BoxCollider2D 추가.

Step 3: BallPrefab 만들기

- Hierarchy에서 “2D Object > Sprites > Circle” 생성, 이름 Ball.
- Add Component로 Rigidbody2D 추가 (→ Gravity Scale = 0으로 변경).
- Add Component로 CircleCollider2D 추가.
- Project 창에서 우클릭 → Create > 2D > Physics Material 2D, 이름 Bouncy, Friction = 0, Bounciness = 1.
- CircleCollider2D의 Material 슬롯에 Bouncy 드래그.
- Ball.cs 스크립트 작성 후 Add Component로 부착.
- Hierarchy의 Ball을 Project 창으로 드래그 → BallPrefab.prefab 생성.
- Hierarchy에 남은 Ball은 삭제 (Prefab만 남기면 됨).

Step 4: GameManager + Spawner

- Hierarchy에 빈 GameObject GameManager 생성.
- BallSpawner.cs 스크립트를 부착.
- Inspector의 Ball Prefab 슬롯에 BallPrefab을 드래그.

Step 5: 재생

- Editor 상단의 Play 버튼 클릭.
- 10개의 공이 무작위로 생성되어 튕기고, 부딪히면 큰 공 하나로 합쳐지는 것을 확인.

8.8. 동작 분석과 확장

田 무엇이 어디서 일어나는가

이벤트	누가 처리하나
공 생성	BallSpawner.Start
초기 속도 설정	Ball.Start
매 프레임 위치 갱신	Rigidbody2D (자동)
벽 충돌 반사	Collider2D + Physics Material 2D (자동)
공-공 충돌 감지	OnCollisionEnter2D 콜백
합치기 계산 및 적용	Ball.Merge
사라진 공 제거	Destroy(other.gameObject)

코드는 약 60줄뿐이지만, 자동 처리되는 부분(렌더링, 물리, 충돌 검출)을 우리가 직접 짤다면 일반 C#/GUI 방식으로 수백 줄이 필요하다.

앞 절의 표가 보여주듯, 우리가 작성한 부분은 “무작위 초기 속도”와 “합치기 로직”뿐이다. 이제 이 코드 골격을 발판으로 학생들이 직접 만들어 볼 도전 과제를 다음 절에서 단계별로 제시한다.

9. 단계별 도전 과제

앞에서 만든 “공 합치기” 예제를 출발점으로 삼아, 학생들이 직접 만들어 볼 수 있는 도전 과제를 난이도 순서대로 제시한다. 모든 과제는 앞 예제의 BallPrefab과 두 스크립트를 그대로 재사용할 수 있게 설계되어 있다.

▲ 과제 진행 권장 순서

- **Phase 1**을 먼저 모두 끝낸다. 코드 수정량이 적고 즉각적인 시각/청각 피드백이 있어 Unity의 “Inspector 즉시 반영” 감각을 익히기 좋다.
- **Phase 2**는 새로운 컴포넌트(UI, AudioSource)나 코루틴이 들어간다. 본문에서 다룬 개념을 실제 코드에서 한 번씩 사용해 보는 단계.
- **Phase 3**는 “공 합치기”와는 다른 게임을 처음부터 만든다. 일반 C#과의 차이를 가장 강하게 체감하는 단계.
- **Phase 4**는 자유 창작. Phase 1~3의 어떤 요소든 조합해 본인만의 게임을 완성한다.

📁 제출 양식 (공통)

- 프로젝트 폴더 압축 — Assets/, Packages/, ProjectSettings/만 포함. Library/, Temp/, Logs/, obj/는 반드시 제외 (용량 폭증).
- README.md — 어떤 과제를 풀었는지, 사용한 키, 화면 캡처 1~2장.
- Demo.mp4 — 30초 이내 동작 영상 (선택, 가능하면).
- Phase별로 최소 1개 과제 이상 풀이를 권장한다.

9.1. Phase 1 — 예제 변형 (기초)

기존 코드의 일부만 바꾸어 시각/청각 피드백을 추가하는 단계이다. “코드를 적게 고쳐도 결과가 크게 달라진다”는 Unity의 강점을 체감해 보자.

🎯 과제 1.1 — 색상 평균으로 합치기

공이 합쳐질 때 새 공의 색이 두 공 색의 평균이 되도록 만들어라.
요구 사항

1. Ball.Start()에서 무작위 색으로 초기화한다.
2. Merge()에서 새 색을 두 공 색의 산술 평균으로 설정한다.

힌트

```
private SpriteRenderer sr;

void Awake() {
    rb = GetComponent<Rigidbody2D>();
    sr = GetComponent<SpriteRenderer>();    // cache
}

void Start() {
    sr.color = new Color(Random.value, Random.value, Random.value);
    /* ... existing velocity init ... */
}

// In Merge():
sr.color = (sr.color + other.sr.color) * 0.5f;
```

생각해 볼 점

Color끼리의 덧셈은 채널별 덧셈이다. 채널 값이 1.0을 넘으면 어떻게 보이는가? “곱셈”은 어떤 의미일까?

🎯 과제 1.2 — 속도에 따라 색이 변하는 공

공의 현재 속력이 클수록 빨강계, 작을수록 파랑계 보이도록 하라.
요구 사항

- Update에서 rb.velocity.magnitude를 읽어 색을 갱신.
- 속도 0~maxSpeed를 0~1로 정규화하여 Color.Lerp 적용.

힌트

```
[SerializeField] private float maxSpeed = 5f;

void Update() {
    float t = Mathf.Clamp01(rb.velocity.magnitude / maxSpeed);
    sr.color = Color.Lerp(Color.blue, Color.red, t);
}
```

확장 합치기 직후 속도가 느려지는 경향(질량 평균)을 색으로 시각화하면 운동량 보존을 눈으로 확인할 수 있다.

🔊 과제 1.3 — 충돌 사운드

벽 충돌과 공-공 합치기에 서로 다른 효과음을 재생하라.
요구 사항

- BallPrefab에 AudioSource 컴포넌트 추가.
- [SerializeField] AudioClip wallHit, mergeHit;로 두 사운드 노출.

- OnCollisionEnter2D에서 상대가 Ball인지 벽인지에 따라 PlayOneShot 분기.

힌트

- 효과음은 .wav 또는 .mp3를 Assets/Audio/ 폴더에 넣고 Inspector 슬롯에 드래그.
- AudioSource.PlayOneShot(clip); — 한 번만 재생, 여러 번 겹쳐 재생 가능.

9.2. Phase 2 — 새 메커니즘 추가 (응용)

새로운 컴포넌트를 도입하거나 게임 규칙 자체를 바꾸는 단계이다. **Coroutine**, **UI 컴포넌트**, **입력 처리**를 직접 사용해 본다.

▶ 과제 2.1 — 마우스 클릭으로 공 추가

화면을 좌클릭하면 그 위치에 새 공이 한 개 생성되도록 하라.
요구 사항

- BallSpawner.Update()에 추가 로직 작성.
- 마우스의 **화면 좌표**(픽셀)를 **월드 좌표**로 변환해야 함.

힌트

```
void Update() {
    if (Input.GetMouseButtonDown(0)) {
        Vector3 mp = Input.mousePosition;
        // distance from camera to z=0 plane
        mp.z = -Camera.main.transform.position.z;
        Vector3 world = Camera.main.ScreenToWorldPoint(mp);
        world.z = 0;
        Instantiate(ballPrefab, world, Quaternion.identity);
    }
}
```

확장 우클릭으로 가장 가까운 공을 삭제하는 기능을 추가해 보라 (Physics2D.OverlapPoint(world)).

✂ 과제 2.2 — 너무 큰 공은 둘로 쪼개기

공의 반지름이 maxRadius 이상이면 둘로 분열되도록 하라.

요구 사항

1. 분열 후 두 공의 **면적의 합** = 원래 공의 면적 ($r_1^2 + r_2^2 = r_{old}^2$, 간단히 $r_1 = r_2 = r_{old}/\sqrt{2}$).
2. 두 공은 서로 **반대 방향으로** 같은 속력으로 출발 (운동량 보존).
3. 무한 분열 방지: 분열 직후 짧은 시간(예: 0.5초) 동안 “막 분열한 공”임을 표시하여 그 사이에 다시 분열되지 않게 한다.

힌트 “막 분열한 공” 플래그는 코루틴으로 관리:

```
private bool justSplit = false;

IEnumerator CooldownAfterSplit() {
    justSplit = true;
    yield return new WaitForSeconds(0.5f);
    justSplit = false;
}
```

```

    }

    void Update() {
        float r = transform.localScale.x * 0.5f;
        if (!justSplit && r > maxRadius) Split();
    }

```

설계 고민 분열된 두 공이 서로 즉시 다시 충돌하면? (거리를 약간 띄워 Instantiate하면 해결)

🔄 과제 2.3 — 코루틴으로 자동 스폰

BallSpawner가 게임 시작 후 2초마다 새 공을 1개씩 자동 추가하도록 하라.

힌트

```

void Start() {
    /* ... initial spawn ... */
    StartCoroutine(SpawnLoop());
}

IEnumerator SpawnLoop() {
    while (true) {
        yield return new WaitForSeconds(2f);
        SpawnOne();
    }
}

```

확장 시간이 지날수록 스폰 간격이 짧아지도록 만들면 난이도 곡선이 생긴다.

🏆 과제 2.4 — HUD 점수 표시

공이 합쳐질 때마다 score를 1씩 올리고, 화면 좌측 상단에 ``Score: 7``처럼 표시하라.
요구 사항

- Hierarchy > Create > UI > Text - TextMeshPro로 텍스트 생성 (자동으로 Canvas 추가).
- GameManager.cs에 public static GameManager Instance로 싱글톤 구현.
- Ball.Merge() 안에서 GameManager.Instance.AddScore(1) 호출.

힌트 싱글톤 골격:

```

public class GameManager : MonoBehaviour
{
    public static GameManager Instance { get; private set; }
    [SerializeField] private TMPro.TextMeshProUGUI scoreText;
    private int score;

    void Awake() {
        if (Instance != null && Instance != this) {
            Destroy(gameObject);
            return;
        }
        Instance = this;
    }

    public void AddScore(int delta) {
        score += delta;
        scoreText.text = "Score: " + score;
    }
}

```

```
}
}
```

일반 C#과의 비교

일반 C#/GUI에서는 “HUD 그리기”를 매 프레임 `g.DrawString`으로 직접 호출해야 했다. Unity에서는 `scoreText.text = ...` 한 줄이면 끝이다 — 언제 그릴지는 Unity의 UI 시스템이 알아서 한다.

9.3. Phase 3 — 새 게임 만들기 (종합 응용)

Rigidbody2D, Collider2D, Prefab, Coroutine, Input을 모두 사용하는 작은 게임을 처음부터 작성한다. 이 단계에서 Unity가 “코드 분량”에서 얼마나 절약되는지를 가장 강하게 체감한다.

■ 과제 3.1 — 벽돌 깨기 (Block Breaker)

“공 합치기” 예제의 `BallPrefab`과 `Bouncy` 머티리얼을 그대로 활용하여 벽돌 깨기 게임을 만들어라.

Hierarchy 설계

Walls	(Top, Left, Right - 3면만 막음)
Bricks	(BrickPrefab을 격자로 배치, 예: 8 x 4)
Paddle	(BoxCollider2D, Rigidbody2D Kinematic)
Ball	(Rigidbody2D + CircleCollider2D + Bouncy, 1개)
GameManager	(점수, 게임 오버 표시)

필수 스크립트

- `Paddle.cs` — 좌우 화살표(또는 A/D)로 패들 이동. `transform.position`을 직접 변경하거나 `rb.velocity.x = h * speed;`를 사용.
- `Brick.cs` — `OnCollisionEnter2D`에서 `Destroy(gameObject)` + 점수 증가.
- `Ball.cs` — 게임 시작 시 위쪽으로 발사. 화면 아래로 떨어지면 (`transform.position.y < -5`) “Game Over”.
- `GameManager.cs` — 모든 Brick이 사라지면 “Stage Clear” (`FindObjectsOfType<Brick>().Length == 0`).

요구 사항

1. 공이 벽돌과 부딪히면 벽돌이 **한 번에** 사라져야 한다 (양쪽이 동시에 부르는 버그 주의).
2. 패들에 부딪힐 때, 부딪힌 위치에 따라 반사 각도가 달라지면 가산점. (벡터를 직접 `rb.velocity`로 설정하면 됨.)
3. HUD에 점수와 남은 벽돌 수를 표시한다.

일반 C#과의 비교 (필수, README에 작성)

- 일반 C#/GUI 방식으로 “공의 벽 반사”와 “벽돌과의 충돌 검사”에 사용했어야 할 코드 줄 수와 본 과제에서 우리가 작성한 줄 수를 세어 비교하라.
- 사라진 코드는 어디로 갔는지 (어떤 컴포넌트가 대신 처리하는지) 한 줄씩 짚지어 설명하라.

🔧 과제 3.2 — 중력 토글 시뮬레이터

“공 합치기” 예제에 중력을 도입하고, SPACE 키로 중력 방향을 토글할 수 있게 하라.
요구 사항

- BallPrefab의 Rigidbody2D.gravityScale = 1.
- GameManager.Update에서 SPACE 입력 시 Physics2D.gravity = -Physics2D.gravity; 토글.
- 합치기 로직은 그대로 유지.

확장 토글 시 모든 공의 색을 일순간 반짝이게 만들어 “중력 방향이 바뀌었음”을 시각적으로 알려라 (코루틴 + SpriteRenderer.color).

9.4. Phase 4 — 자유 창작**💡 과제 4 — 자신만의 미니 게임**

지금까지 배운 Unity 개념을 **최소 5개 이상** 사용하여, 자유 주제로 미니 게임을 한 편 완성 하라.

필수 사용 개념 (다음 중 5개 이상 명시적으로 사용)

- [SerializeField]로 Inspector 노출 변수
- Awake에서 GetComponent 캐싱
- Rigidbody2D 또는 Collider2D
- Prefab + Instantiate
- OnCollisionEnter2D 또는 OnTriggerEnter2D
- Coroutine (IEnumerator, yield return)
- Time.deltaTime으로 프레임 무관 이동
- TextMeshProUGUI로 HUD

주제 예시 (자유롭게 변경 가능)

- **Catch the Star** — 위에서 별이 떨어지고 플레이어가 받는 게임
- **Mini Shooter** — 위쪽 적이 내려오고 아래 플레이어가 발사
- **Dodge** — 떨어지는 공을 좌우로 피하는 게임
- **Top-Down Maze** — 미로 안에서 골인 지점 찾기
- **Flappy 스타일** — 클릭으로 점프하며 장애물 피하기

평가 기준

1. 동작 — 에러 없이 Play 가능, 1분 이상 플레이 가능.
2. Unity 활용 — 위 개념 중 5개 이상을 실제로 사용 (README에 사용 위치 명시).
3. 비교 글쓰기 — “이 게임을 일반 C#/GUI 방식(Timer + Paint 이벤트)으로 짰다면 어디가 더 길어질지” 한 단락으로 README에 기술.
4. 창의성 — 위 예시와 다른 주제 또는 메커니즘 한 가지 이상 추가 시 가산점.

팁

- 무리하게 큰 게임을 시작하지 말고, “한 가지 메커니즘”만 잘 동작하게 만든 후 확장하라.
- Phase 1~3에서 만든 자산(Prefab, 머티리얼, 사운드)을 그대로 가져다 써도 좋다.

10. 이번 강의에서 배운 핵심 개념

개념	핵심 내용
MonoBehaviour 상속 생명 주기 메서드	Unity가 인식하는 스크립트가 되기 위한 필수 조건 Awake/Start/Update/FixedUpdate/LateUpdate의 호출 시점
컴포넌트 모델 GetComponent 캐싱 Time.deltaTime	상속이 아닌 조합으로 GameObject의 기능을 구성 Update에서 매번 호출하지 말고 Awake에서 한 번만 프레임 속도와 무관한 일정 속도 이동
Inspector 직렬화 new 금지 가짜 null Coroutine	public / [SerializeField] 규칙 MonoBehaviour는 AddComponent 또는 Instantiate 로 파괴된 Unity 오브젝트는 == null 로만 안전하게 검사 시간 흐름이 있는 작업을 간결하게 표현

☞ 마무리 도전 과제

1. Unity Editor에서 빈 GameObject를 만들고, LifecycleDemo 스크립트를 붙여 **Console** 창에서 Awake → Start 순서를 직접 확인하라.
2. Mover 스크립트를 작성하여 speed를 Inspector에서 조정하며 오브젝트가 다른 속도로 움직이는 것을 확인하라.
3. “화살표 키로 움직이는 캐릭터”를 Unity에서 작성해 보라. Player.cs의 키 입력 + transform.position 변경으로 충분하다.
4. Bomb 코루틴을 변형하여, 1초 간격으로 Debug.Log를 5번 출력한 후 오브젝트가 스스로 사라지도록 만들라.